1. VM Specification Part II: Program Control and Functions

1.1 Overview

The **Virtual Machine (VM)** acts as a **bridge** between the **high-level language (Jack)** and the **low-level Hack assembly language**.
Part II introduces **program control** and **function calling mechanisms** (the higher-level logic of computation).

1.2 Program Control Commands

Used to control the flow of execution.

Command	Syntax	Purpose
`label labelName`	Declares a label	Used for marking a place in code for jumps.
`goto labelName`	Unconditional jump	Transfers control to labelName.
`if-goto labelName`	Conditional jump	Jumps if top of stack ≠ 0.

Example:

```
// VM code
if-goto LOOP_START
goto END
label LOOP_START
```

Translation Concept:
Each of these translates to **Hack assembly labels** and **jump instructions (JMP, JGT, JEQ, etc.)**.

1.3 Function Calling Commands

Command	Syntax	Description
`function f nVars`	Defines a function	Declares function `f` with `nVars` local variables.
`call f nArgs`	Calls function	Invokes function `f` with `nArgs` arguments.
`return`	Returns from function	Restores caller's state and continues after call.

1.4 The Call Stack Mechanism

Every function call uses a **stack frame** to store:

1. The return address
2. Caller's `LCL`, `ARG`, `THIS`, `THAT`
3. Function arguments
4. Local variables

Call sequence:

1. Push return address
2. Save caller segments
3. Reposition `ARG`
4. Reposition `LCL`
5. Transfer control to called function
6. When returning, restore caller segments and jump to return address

---

1.5 Example – Function Call Translation

```
call sum 2
```

→ (in assembly)

```
push returnAddress
```

```
push LCL
push ARG
push THIS
push THAT
ARG = SP - 5 - nArgs
LCL = SP
goto functionName
(returnAddress)
```

1.6 Perspective

- The **VM abstraction** hides the hardware details.
- Enables writing portable high-level programs (Jack).
- Stack-based model is simple and language-independent.
- Provides the **foundation for the Jack compiler** in the next stage.

---

2. VM Implementation (Part II)

2.1 Purpose

Implementing the second half of the **VM translator**, which handles:

- Program flow commands (`label`, `goto`, `if-goto`)
- Function commands (`function`, `call`, `return`)

2.2 Structure of the Translator

The **VM translator** converts `.vm` files into `.asm` files.

Components:

1. **Parser** – Reads and classifies VM commands.
2. **CodeWriter** – Generates Hack assembly code for each VM command.
3. **Main Program** – Controls translation of multiple `.vm` files.

2.3 Implementation Details

a. Program Flow Implementation

- Each label is translated as `(functionName$labelName)` to make it unique across functions.
- `goto` and `if-goto` commands translate into `@label` followed by `0;JMP` or conditional jump (`D;JNE`).

b. Function Handling

- `function` command creates local variables (initialized to 0).
- `call` sets up the new stack frame (return address, etc.).
- `return` restores the previous frame and jumps to return address.

2.4 Bootstrap Code

When the program starts, the **translator**:

1. Sets `SP = 256`
2. Calls `Sys.init` to start execution

2.5 Testing and Verification

- Each VM file (e.g., SimpleFunction.vm, Fibonacci.vm) is translated and tested using the Hack platform.
- Correct output confirms that stack frames and control flow are correctly implemented.

2.6 Perspective

- This level completes the **Virtual Machine abstraction**.
- From now on, the system can fully execute **Jack programs** by compiling them into VM code and running them on the **VM implementation**.

---

3. High-Level Language: Background

3.1 Motivation

To make programming easier and human-readable, a **high-level language** (Jack) is built on top of the VM abstraction.

- Assembly is too detailed and hardware-specific.
- High-level language hides memory management and control logic.
- Enables **abstraction, modularity, and reusability.**

3.2 Compilation Path (Full System)

```
Jack Program
    ↓
Jack Compiler → VM Code
    ↓
VM Translator → Hack Assembly
    ↓
Assembler → Binary Machine Code
    ↓
Hardware Simulation
```

---

4. The Jack Language Specification

4.1 Overview

Jack is a simple, Java-like high-level language designed for education and system building.

4.2 Basic Features

- Object-oriented
- Strongly typed

- Supports classes, methods, constructors
- Simple syntax close to Java/C++

4.3 Jack Program Structure

A **Jack program** consists of one or more **class files**.

Example:

```
class Main {
    function void main() {
        do Output.printString("Hello World");
        return;
    }
}
```

4.4 Components

Element	Description
Class	Basic unit of Jack program
Fields/Static vars	Class-level data
Methods/Constructors/Functions	Behavior of the class
Statements	let, if, while, do, return
Expressions	constants, operators, variables

4.5 Data Types

- **Primitive:** `int`, `char`, `boolean`
- **Reference:** `className`
- **Array** type: e.g., `Array a`

4.6 Subroutines

Type	Usage
`function`	Used for computations (no object context)
`method`	Operates on object instance
`constructor`	Creates new object

4.7 Statements

- `let x = expression;`
- `if (condition) { statements } else { statements }`
- `while (condition) { statements }`
- `do subroutineCall;`
- `return expression;`

4.8 Jack Standard Library

Jack includes built-in classes to handle:

- **Input/Output:** `Output`, `Keyboard`
- **Math:** `Math`
- **Memory:** `Memory`
- **String, Array**, etc.

---

5. Writing Jack Applications

5.1 Development Process

1. Write `.jack` source code.
2. Compile using **Jack Compiler** → generates `.vm` file.
3. Run `.vm` on **VM Emulator** or translate to assembly and execute on Hack CPU.

5.2 Example Program

```
class Main {
   function void main() {
      var int i;
      let i = 0;
      while (i < 10) {
         do Output.printInt(i);
         let i = i + 1;
      }
      return;
   }
}
```

Process:

- Compiler breaks into VM instructions.
- VM code executes using stack-based commands.
- Translated assembly runs on the Hack CPU.

5.3 Advantages of Jack

- Easy to learn, Java-like syntax.
- Demonstrates full compilation chain.
- Supports object-oriented and modular design.

- Bridges gap between software and hardware.

5.4 Perspective

This stage completes the **entire computer system hierarchy**:

```
Hardware (NAND gates)
↓
Hack CPU & Memory
↓
Assembly Language
↓
Virtual Machine
↓
Jack Language & Compiler
↓
Jack Applications
```

✅ Summary Table

Level	Description	Example
Hardware	Hack computer built from logic gates	ALU, Memory, CPU
Assembly	Low-level Hack instructions	`@i`, `D=M`
VM	Stack-based intermediate code	`push constant 5`, `call f 2`
Jack	High-level language	`do Output.printInt(x);`
Application	User program	Games, animation, calculator